

UNIT-1

LEET CODE EXERCISE

1. Big Countries

Problem Summary

You are given a table World with the following columns:

- name (country name)
- continent
- area (in square km)
- population
- gdp

A country is considered “big” if:

- Its area $\geq 3,000,000$, OR
- Its population $\geq 25,000,000$

Task

Write an SQL query to return the name, population, and area of all big countries.

SQL Solution

```
</> Code
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT name, population, area FROM World WHERE area >= 3000000 OR population >= 25000000;

Saved Ln 1, Col 1
```

Testcase Test Result

Accepted Runtime: 82 ms

Case 1

Input

World =

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

Output

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

The SELECT clause retrieves required columns.

The WHERE clause filters records based on the given conditions.

The OR operator ensures that even if one condition is satisfied, the country is included.

2. Find Customer Referee

Problem Summary

You are given a table Customer with the following columns:

- id (customer ID)
- name (customer name)
- referee_id (ID of the referring customer)

Task

Find the names of customers who were NOT referred by the customer with id = 2.

- ☐ Note: Customers with NULL in referee_id should also be included.

SQL Solution

```
</> Code
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT name FROM Customer WHERE referee_id != 2 OR referee_id IS NULL;
```

Saved Ln 1, Col 1

Testcase Test Result

Accepted Runtime: 95 ms

Case 1

Input

Customer =

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

Output

name
Will
Jane
Bill
Zack

referee_id != 2 → selects customers not referred by ID 2
referee_id IS NULL → includes customers with no referee
OR ensures both conditions are considered

3. Not Boring Movies

Problem Summary

You are given a table Cinema with the following columns:

- id (movie ID)
- movie (movie name)
- description (movie type)
- rating (movie rating)

Task

Find all movies that are:

- Not boring → description != 'boring'
- Have odd-numbered IDs → id % 2 = 1

Return the result sorted by rating in descending order.

SQL Solution



The screenshot shows a SQL IDE with a code editor and a test result panel. The code editor contains a MySQL query. The test result panel shows the query was accepted and displays the input and output tables.

```
1 # Write your MySQL query statement below
2 SELECT * FROM Cinema WHERE description != 'boring' AND id % 2 = 1 ORDER BY rating DESC;
```

Accepted Runtime: 98 ms

Case 1

Input

id	movie	description	rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	irish	boring	6.2
4	Ice song	Fantasy	8.6
5	House card	Interesting	9.1

Output

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

description != 'boring' → filters out boring movies

id % 2 = 1 → selects only odd-numbered IDs

ORDER BY rating DESC → sorts results from highest to lowest rating

4. Invalid Tweets

Problem Summary

You are given a table Tweets with the following column:

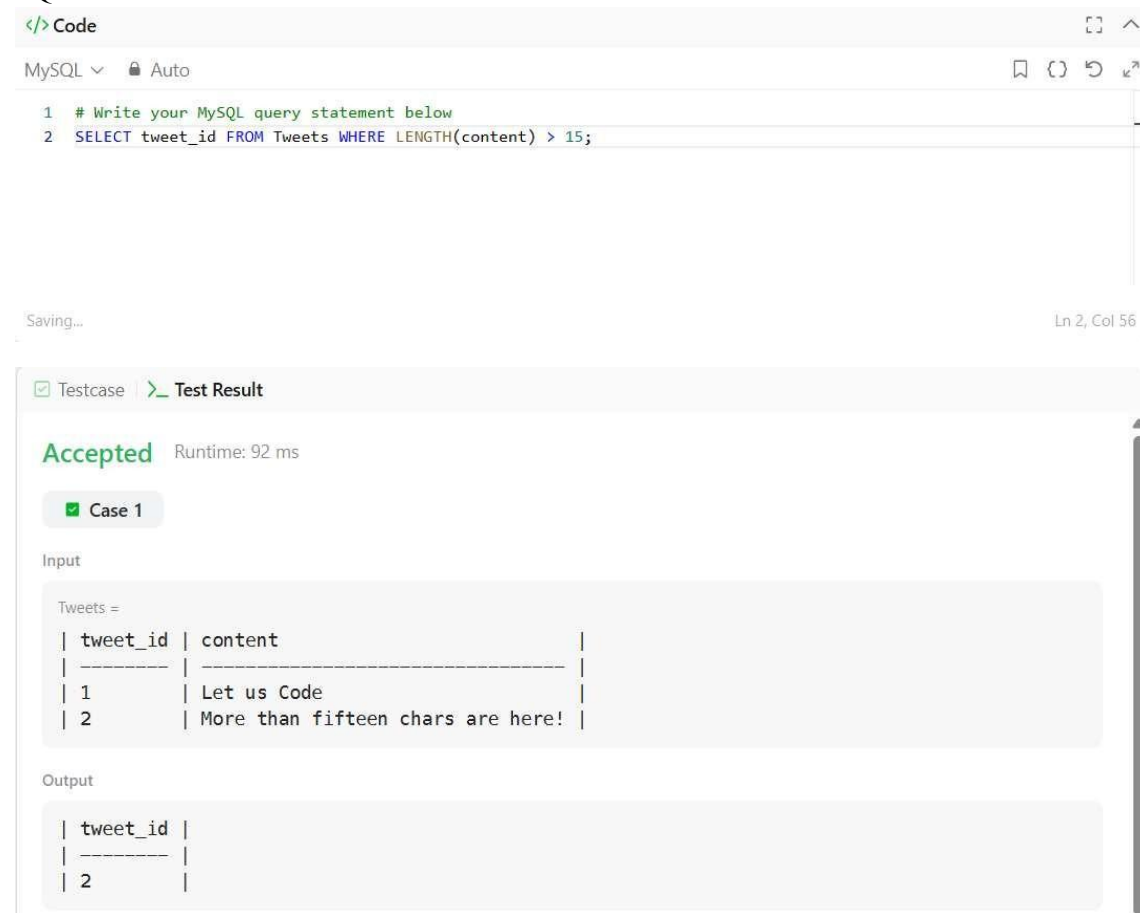
- tweet_id (ID of the tweet)
- content (text of the tweet)

Task

Find the IDs of tweets that are invalid, where a tweet is considered invalid if:

- ☐ The length of the content is greater than 15 characters

SQL Solution



The screenshot shows a SQL IDE interface. At the top, there's a 'Code' editor with a MySQL query: `SELECT tweet_id FROM Tweets WHERE LENGTH(content) > 15;`. Below the code editor, there's a 'Testcase' tab showing 'Case 1' as 'Accepted' with a runtime of 92 ms. The input data is a table with two rows: (1, 'Let us Code') and (2, 'More than fifteen chars are here!'). The output is a table with one row: (2, 'More than fifteen chars are here!').

```
1 # Write your MySQL query statement below
2 SELECT tweet_id FROM Tweets WHERE LENGTH(content) > 15;
```

Accepted Runtime: 92 ms

Case 1

Input

Tweets =

tweet_id	content
1	Let us Code
2	More than fifteen chars are here!

Output

tweet_id
2

LENGTH(content) calculates the number of characters in the tweet
15 filters tweets that exceed the allowed length
Only tweet_id is returned as required

5. Recyclable and Low-Fat Products

Problem Summary

You are given a table Products with the following columns:

- product_id
- low_fats (values: 'Y' or 'N')
- recyclable (values: 'Y' or 'N')

Task

Find the product IDs of products that are:

- Low fat (low_fats = 'Y')
- Recyclable (recyclable = 'Y')

SQL Solution

The screenshot shows a SQL IDE interface. The top panel, titled 'Code', contains a MySQL query: `SELECT product_id FROM Products WHERE low_fats = 'Y' AND recyclable = 'Y';`. The bottom panel, titled 'Test Result', shows the query was 'Accepted' with a runtime of 87 ms. It displays 'Case 1' with input and output tables.

Input

Products =

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

Output

product_id
1
3

`low_fats = 'Y'` → selects only low-fat products

`recyclable = 'Y'` → selects recyclable products

`AND` ensures both conditions are satisfied

6. Article Views

Problem Summary

You are given a table Views with the following columns:

- `article_id`
- `author_id`
- `viewer_id`
- `view_date`

Task

Find all authors who viewed at least one of their own articles.

- Return the result as a list of unique author IDs
- Sort the result in ascending order

SQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 Select distinct author_id as id from Views where author_id = viewer_id order by id;
```

Ln 2, Col 2

Testcase Test Result

Accepted Runtime: 71 ms

Case 1

Input

Views =

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21

Output

id
4
7

Explanation

- `author_id = viewer_id` → identifies cases where authors viewed their own articles
- `DISTINCT` → removes duplicate author IDs
- `ORDER BY id` → sorts the result in ascending order